Mochi Desk

Animated desktop companion firmware for the ESP32-C3 SuperMini

Target board	ESP32-C3 SuperMini
Display	SSD1306 128×64 monochrome OLED, I2C 0x3C
Animations	18 clips, 1383 frames, ~14 FPS
Frame storage	231 KB compressed (from 1383 KB raw)
Toolchain	Arduino IDE, Adafruit SSD1306 + GFX
Document version	1.0

Contents

1 What you have

The package is a single Arduino sketch folder. Everything needed to build and flash the gadget is inside it; nothing is fetched at compile time except the two Adafruit libraries.

File	Purpose	Size
MochiDesk.ino	The whole program: decoder, playback engine, touch handling, sleep	~11 KB
animations.h	Index of all 18 clips and the ANIMS[] lookup table	~3 KB
anim_*.h (18 files)	Compressed frame data, one file per animation	~231 KB total
tools/gif2mochi.py	Encoder that turns any GIF into one of those headers	~7 KB
README.md	Short quick-start version of this document	~2 KB

The folder name and the .ino name must stay identical — that is how the Arduino IDE finds a sketch. If you rename one, rename both.

2 Hardware and wiring

These pin assignments were confirmed with an I2C scan on the actual board, not read off a diagram. The scan found the display on SDA 20 / SCL 21, which is the reverse of what the original wiring diagram showed.

Signal	GPIO	Notes
OLED SDA	20	I2C data
OLED SCL	21	I2C clock
OLED VDD / GND	3.3 V / GND	Do not run the display from 5 V
TTP223 I/O	3	Active HIGH, momentary mode
Buzzer / speaker +	4	See the caution below

Two cautions

- **GPIO 20 and 21 are UART0.** With *USB CDC On Boot* disabled, the Arduino core takes those pins for the serial port and fights the I2C bus, and the display stays blank. Leave that setting enabled, or move the OLED to GPIO 5 and 6 and change the two defines at the top of the sketch.
- **Do not drive an 8 Ω speaker straight from GPIO 4.** It exceeds what the pin can source. Use a small NPN transistor, a dedicated amplifier module, or swap in a piezo buzzer.

Power

The TP4056 and LiPo from the original build still work here. Feed the board on 3.3 V from the charger output if your module has no regulator, or on 5 V only if the board carries its own LDO. A 110 mAh cell will not last long: the OLED plus continuous frame decoding draws steadily, and there is no sleep-to-save-power mode in this firmware, only a dimmed idle animation. Budget a larger cell if you want all-day runtime.

3 Installation

Step 1 — put the folder in place

Unzip `MochiDesk.zip` into your Arduino sketchbook, then open `MochiDesk.ino`. The 18 animation headers must sit beside the .ino in the same folder; the IDE opens them as extra tabs.

Step 2 — libraries

Library Manager: **Adafruit SSD1306** and **Adafruit GFX Library**. Nothing else. There is no networking in this sketch.

Step 3 — board settings

Setting	Value	Why it matters
Board	ESP32C3 Dev Module	SuperMini variant also works
USB CDC On Boot	Enabled	Frees GPIO 20/21 for I2C; also gives you serial
Partition Scheme	Default 4MB with spiffs	231 KB of frames fits inside 1.3 MB
Flash Mode	QIO	Switch to DIO if the board fails to boot
Upload Speed	921600	Drop to 115200 if uploads fail

Step 4 — flash and check

Compilation takes noticeably longer than a normal sketch because the compiler is chewing through 231 KB of hex literals. On the first boot you should see:

- one short beep as the sketch starts
- the 105-frame Intro clip playing once, about 7.4 seconds
- the display settling into the looping Happy animation
- serial output at 115200 listing the animation count

If instead you get one long low beep and a blank screen, the display was not found at 0x3C on GPIO 20/21. Run the diagnostic sketch in Appendix B.

4 Using the gadget

Input	Action
Single tap	Next animation in order
Double tap	Random animation (never picks the Intro)
Long press, 1.5 s	Toggle shuffle mode; high beep on, low beep off
No touch for 5 minutes	Dims the screen and plays Sleepy
Any tap while asleep	Wakes up and plays Excited

In shuffle mode the gadget picks a new random clip each time the current one finishes a full loop, so it changes roughly every five seconds. The mode survives until you long-press again or reset the board; it is not saved to flash.

Serial console

Open the Serial Monitor at 115200 with line ending set to Newline:

Type	Result
a number, 0 to 17	Jump straight to that animation
list	Print every animation with its index and frame count
shuffle	Toggle shuffle mode

Options you can change

All near the top of the .ino:

Define	Default	Effect
FRAME_MS	70	Milliseconds per frame. 70 = 14 FPS, the original rate. Lower is faster.
SOUND_ON	true	Set false to silence all beeps
SHOW_NAME	true	Flash the emotion name across the bottom on a change
SHUFFLE_ON_BOOT	false	Start in shuffle mode
SLEEP_AFTER_MS	5 min	Idle timeout before the sleepy animation
I2C_HZ	700000	Bus speed. Drop to 400000 if the screen tears or glitches.

5 Animation reference

One representative frame from each clip, in the order the single-tap button cycles through them.

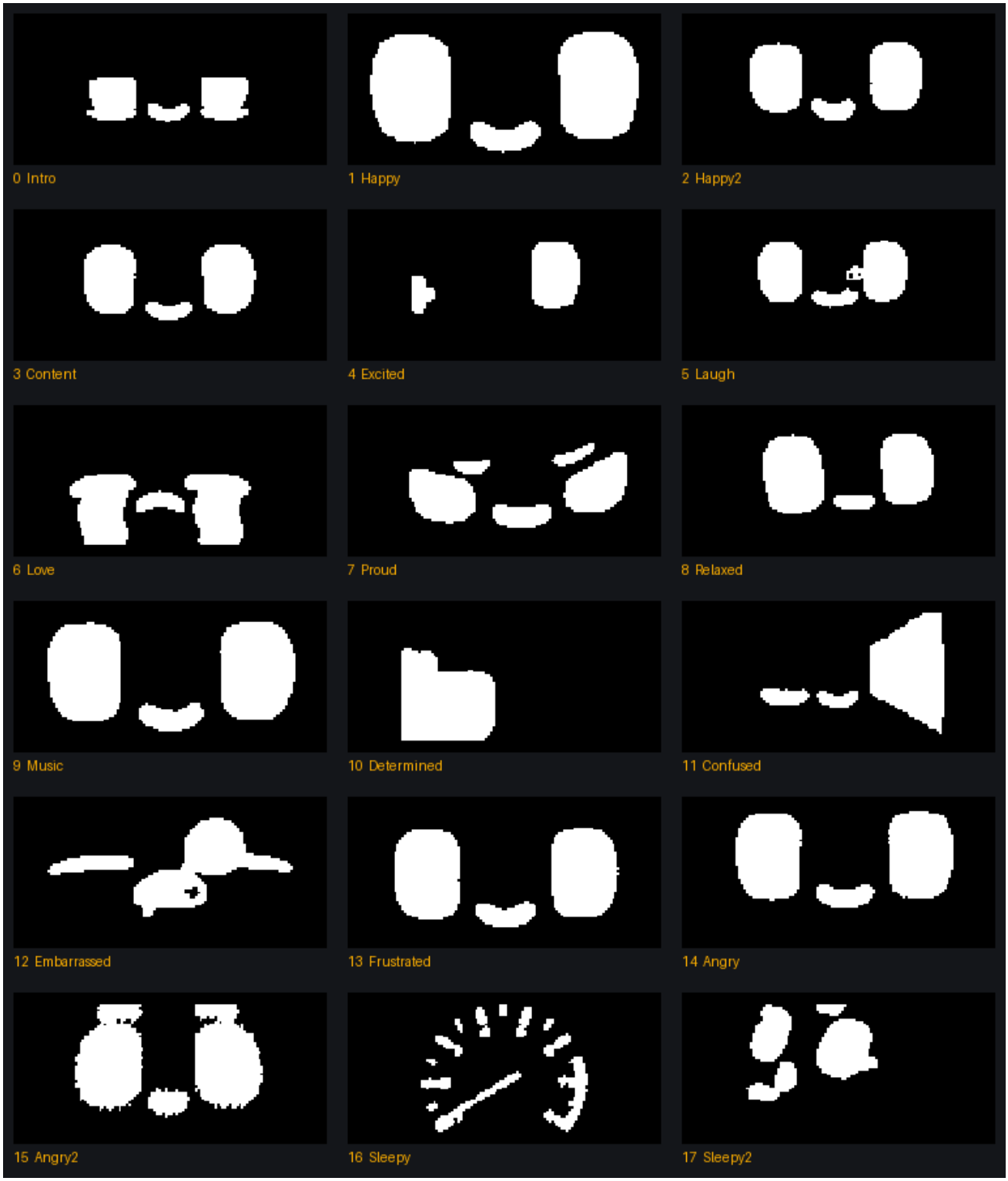

Figure 1 — all 18 animations as they render on the 128×64 display.

Every clip runs at 70 ms per frame. Duration is frames × 70 ms.

#	Name	Frames	Duration	Raw	Stored	Ratio
0	Intro	105	7.3 s	105 KB	8.1 KB	13.0×
1	Happy	75	5.2 s	75 KB	8.7 KB	8.7×
2	Happy2	75	5.2 s	75 KB	9.9 KB	7.6×
3	Content	76	5.3 s	76 KB	12.9 KB	5.9×
4	Excited	75	5.2 s	75 KB	8.1 KB	9.3×
5	Laugh	75	5.2 s	75 KB	9.8 KB	7.7×
6	Love	75	5.2 s	75 KB	11.0 KB	6.8×
7	Proud	75	5.2 s	75 KB	14.5 KB	5.2×
8	Relaxed	75	5.2 s	75 KB	13.3 KB	5.6×
9	Music	75	5.2 s	75 KB	31.2 KB	2.4×
10	Determined	75	5.2 s	75 KB	10.5 KB	7.1×
11	Confused	76	5.3 s	76 KB	14.0 KB	5.4×
12	Embarrassed	75	5.2 s	75 KB	8.7 KB	8.6×
13	Frustrated	75	5.2 s	75 KB	12.2 KB	6.2×
14	Angry	75	5.2 s	75 KB	12.1 KB	6.2×
15	Angry2	75	5.2 s	75 KB	20.3 KB	3.7×
16	Sleepy	76	5.3 s	76 KB	8.8 KB	8.6×
17	Sleepy2	75	5.2 s	75 KB	17.4 KB	4.3×
	TOTAL	**1383**	**97 s**	**1383 KB**	**231 KB**	**6.0×**

Music compresses worst, at 2.5×, because its equalizer bars are dithered noise that changes across the whole screen every frame — exactly the case delta encoding cannot help with. Happy compresses best, at 9×, because only the eyelids move.

6 How a frame is stored

Understanding this section is what lets you add your own clips or debug a corrupted one. There are three layers stacked on top of each other.

Layer 1 — the raw bitmap

A frame is 128 × 64 pixels, one bit per pixel: 128 × 64 ÷ 8 = **1024 bytes**. The layout is row-major with the most significant bit first, which is exactly the format `Adafruit_GFX::drawBitmap` expects, so no conversion is needed at draw time. The pixel at (x, y) lives at:

```
byte  = buffer[ y * 16 + (x >> 3) ]
bit   = 0x80 >> (x & 7)
lit   = byte & bit

// 16 = 128 pixels / 8 bits per byte
```

Layer 2 — XOR delta

Storing 1383 frames of that would cost 1.4 MB. But consecutive frames of a blinking face are nearly identical, so what gets stored is not the frame — it is the difference from the frame before, computed as a byte-wise XOR:

```
delta[i] = frame[i] XOR previous_frame[i]

// decoding is the same operation, which is why the decoder
// XORs into the buffer instead of overwriting it:
//     frameBuf[o] ^= decoded_byte
```

Unchanged regions become runs of 0x00, which the next layer crushes. The cost of this trick is that **frames must be played in order from frame 0 with a cleared buffer**. Jump into the middle of a clip and you get garbage, because the chain of differences has no starting point. `startAnim()` exists to enforce that.

Layer 3 — PackBits run-length coding

PackBits is a classic byte-oriented RLE. Each token byte is read as a *signed* value:

Token (signed)	Meaning
0 to 127	The next (token + 1) bytes are literals — copy them out as they are
–1 to –127	The next single byte repeats (1 – token) times, so 2 to 128 copies
–128	Never emitted by this encoder

A worked example. Suppose a delta begins with 300 unchanged bytes, then the two bytes 0x3C 0x18, then 60 more unchanged bytes:

```
raw    : 00 00 00 ... (300 times) ... 3C 18 00 00 ... (60 times)

encoded: 81 00        <- 0x81 = -127 signed, so repeat next byte 128 times
         81 00        <- another 128
         D3 00        <- 0xD3 = -45,  repeat 46 times   (128+128+46 = 302)
         01 3C 18     <- 0x01 = +1,   two literal bytes follow
         C5 00        <- 0xC5 = -59,  repeat 60 times

362 bytes become 11.
```

Layer 4 — the file format

Each `anim_*.h` holds two arrays. One blob contains every compressed frame back to back; an offset table says where each frame starts. Frame *n* occupies bytes `offsets[n]` up to `offsets[n+1]`.

```
#define HAPPY_FRAMES 75

const uint16_t happy_offsets[76] PROGMEM = { 0, 142, 260, 371, ... };
const uint8_t  happy_data[8623]  PROGMEM = { 0x81, 0x00, 0x81, ... };
```

The offset table is `uint16_t`, so a single animation cannot exceed 65535 compressed bytes. The largest here, Music, uses 32 KB, so there is plenty of headroom — but the encoder checks and refuses rather than silently wrapping.

7 The conversion pipeline

This is what turned the 18 source GIFs into the headers. The same steps run inside `tools/gif2mochi.py`, so you can repeat them on any GIF of your own.

Step	What happens	Why
1 Load	Every GIF frame is converted to 8-bit greyscale	The sources are phone recordings of a screen, so they carry colour fringing that is just noise
2 Denoise	3×3 median filter	Kills speckle that would otherwise survive thresholding
3 Auto-crop	The bright area is measured across the whole clip, padded, and forced to a 2:1 box	Each GIF was framed differently; this centres the face and matches the display aspect
4 Resize	Lanczos down to 128×64	Smooth downscale before the hard threshold
5 Threshold	Pixels above 110 become white	Turns the soft OLED glow into clean 1-bit pixels
6 Pack	Bits packed row-major, MSB first, into 1024 bytes	Matches drawBitmap
7 Delta	XOR against the previous frame	Turns static regions into zeros
8 Compress	PackBits	Collapses those zero runs

Results

Stage	Size	Note
18 source GIFs	23.6 MB	480×304 colour
Raw 1-bit frames	1383 KB	Would not fit the 1.3 MB app partition alongside code
After PackBits only	593 KB	2.3×
After XOR delta + PackBits	231 KB	6.0× — what ships

The delta step is doing most of the work. On its own PackBits only manages 2.3×, because a single face frame has plenty of edges; the moment you encode differences instead, most of each frame becomes zeros.

8 Code walkthrough

The decoder

This is the heart of the sketch. It reads one frame's compressed bytes out of flash and XORs the result into the persistent frame buffer.

```
void decodeFrame(const Anim& a, uint16_t frame) {
  uint16_t i   = pgm_read_word(&a.offsets[frame]);      // start of this frame
  uint16_t end = pgm_read_word(&a.offsets[frame + 1]);  // start of the next one
  uint16_t o   = 0;                                     // output byte counter

  while (o < 1024 && i < end) {
    int8_t t = (int8_t)pgm_read_byte(&a.data[i++]);     // read the token
    if (t >= 0) {                                       // literal run
      uint16_t n = (uint16_t)t + 1;
      while (n-- && o < 1024) frameBuf[o++] ^= pgm_read_byte(&a.data[i++]);
    } else {                                            // repeat run
      uint16_t n = (uint16_t)(1 - t);
      uint8_t  v = pgm_read_byte(&a.data[i++]);
      while (n-- && o < 1024) frameBuf[o++] ^= v;
    }
  }
}
```

Three details worth noticing. The cast to `int8_t` is what makes 0x81 read as −127 rather than 129. The `o < 1024` guard means a corrupted header can never write past the buffer. And `^=` rather than `=` is the delta step — remove it and you get noise.

Starting an animation

```
void startAnim(uint8_t idx, bool once, uint8_t thenIdx) {
  animIdx  = idx % ANIM_COUNT;
  frameIdx = 0;
  memset(frameBuf, 0, sizeof(frameBuf));   // the delta chain restarts from black
  ...
}
```

The `memset` is mandatory, not tidiness. Every path that changes animation goes through this function for that reason, including the loop-back at the end of a clip.

The playback tick

```
void tickAnim() {
  if (millis() - lastFrame < FRAME_MS) return;   // non-blocking timing
  lastFrame = millis();

  decodeFrame(ANIMS[animIdx], frameIdx);
  drawCurrent();

  frameIdx++;
  if (frameIdx >= ANIMS[animIdx].frames) {
    if (playOnce)     startAnim(nextAfterOnce);  // intro hands over to Happy
    else if (shuffle) startAnim(randomOther());
    else              startAnim(animIdx);        // loop, resetting the chain
  }
}
```

There is no `delay()` anywhere in the playback path. That is the main structural difference from the repos this was derived from, whose `playGIF()` blocks for the whole clip and cannot read the touch pad while animating.

Where the time goes

Operation	Roughly	Note
decodeFrame	under 1 ms	A few thousand byte operations
drawBitmap	1 to 2 ms	8192 drawPixel calls into the GFX buffer
display.display()	12 to 25 ms	1024 bytes over I2C; the real bottleneck
Frame budget	70 ms	Comfortable headroom at 700 kHz

At 400 kHz the I2C transfer alone is about 23 ms, which still fits 70 ms. If you push FRAME_MS below about 35 you will hit the bus ceiling and the animation will simply run at whatever rate the display can accept.

The touch state machine

`serviceTouch()` distinguishes three gestures from one digital pin. A press longer than 1500 ms fires immediately as a long press and sets a flag so the release is ignored. Otherwise each release increments a tap counter; once 320 ms pass with no further tap, one tap resolves as *next* and two or more as *random*. That 320 ms is the deliberate cost of supporting a double tap — a single tap cannot act sooner without ruling out the second one.

9 Verification

The decoder was not assumed to work. It was tested before shipping:

- The exact `decodeFrame()` from the sketch was compiled as plain C on a desktop, with `pgm_read_byte` and `pgm_read_word` stubbed to plain dereferences, and run against the generated headers.
- It decoded 255 frames across three clips (Happy, Music, Intro) into 261,120 bytes, which were compared byte for byte against the reference bitmaps produced directly from the GIFs. **Bit-exact match.**
- Decoded frames were rendered back to images and inspected visually, to catch the case where encoder and decoder agree on something that is nonetheless wrong.
- `tools/gif2mochi.py` was run on love.gif and its output compared against the shipped `anim_love.h`: identical, all 11,247 bytes. The tool you have is the tool that made these files.

What has *not* been tested is the sketch running on physical hardware, since that is not something I can do from here. The timing figures in section 8 are calculated from bus speed and instruction counts, not measured.

10 Adding your own animations

Step 1 — prepare a GIF

Anything works, but the result is 1-bit black and white on a 2:1 screen, so high-contrast material with a dark background reads best. Keep it under about 250 frames.

Step 2 — encode it

```
pip install pillow
python3 tools/gif2mochi.py mysad.gif sad Sad

# useful switches:
#   --threshold 130    raise if the result is too white, lower if too dark
#   --no-autocrop      keep the original framing instead of finding the content
#   --no-denoise       skip the median filter on clean source art
```

The tool prints the compression ratio, runs its own round-trip decode check, and tells you the two lines to paste next.

Step 3 — register it

Drop `anim_sad.h` next to the others and add two lines to `animations.h`:

```
#include "anim_sad.h"                                    // with the other includes

const Anim ANIMS[] = {
  ...
  { "Sad        ", sad_data, sad_offsets, SAD_FRAMES },  // add this row
};
```

`ANIM_COUNT` recalculates itself from the array size, so nothing else needs touching. The name field is padded to 11 characters only so the on-screen label lines up; it is cosmetic.

Memory budget

	Used	Available	Headroom
Flash (app partition)	~521 KB	1310 KB	about 1 MB
RAM	~2 KB for buffers	~320 KB	ample

Roughly 1 MB of flash is free, which at the observed 6× ratio is another 6000 frames or so. If you ever exceed it, switch the partition scheme to *Huge APP (3MB No OTA)* for 3 MB of program space.

11 Troubleshooting

Symptom	Most likely cause	Fix
Blank screen, one long low beep at boot	Display not answering at 0x3C on GPIO 20/21	Run the Appendix B scanner. Check SDA/SCL are not swapped, and that VDD is on 3.3 V.
Blank screen, no beeps, nothing on serial	USB CDC On Boot is disabled, so the serial port is fighting the I2C pins	Enable it in Tools, or move I2C to GPIO 5 and 6.
Animation shows garbage or ghost pixels	The delta chain was broken — playback started somewhere other than frame 0	Every animation change must go through startAnim(). Do not set animIdx directly.
Screen tears or the image flickers	I2C running faster than your module tolerates	Set I2C_HZ to 400000.
Animation runs slower than 14 FPS	I2C transfer is the ceiling	Raise I2C_HZ, or accept it; the timing is non-blocking so nothing else breaks.
Touch fires twice or not at all	TTP223 jumper set to toggle mode, or a wiring issue	Confirm the pad reads 1 while touched using the Appendix B sketch.
Compile fails: 'section .rodata will not fit'	Too much frame data for the partition	Switch to Huge APP (3MB No OTA), or remove some animations.
Compile fails on an identifier starting with a digit	A hand-made header named something like 225frames_data	C identifiers cannot start with a digit. Rename it.

Appendix A Companion sketch: Info Terminal

Separate firmware for the same board, from earlier in this project. It shares the pin map, so you can flash either one onto the same hardware.

Screen	Shows	Source
Clock	Time, seconds, weekday, date	NTP, timezone IST-5:30
Weather	Temperature, condition icon, min/max, humidity, wind	ip-api for location, Open-Meteo for data; no API keys
Crypto	BTC, ETH, SOL in USD with 24 h change	CoinGecko; no API key
Notes	Your own text, wrapped to 21 characters and auto-paged	Typed into the built-in web page, stored in NVS

Same touch pad: tap cycles the four screens, long press forces a data refresh. It hosts a small web page at `http://terminal.local` for editing the notes and the Wi-Fi credentials, and falls back to its own hotspot (`InfoTerminal` / `12345678`, at 192.168.4.1) if it cannot join a network. Extra libraries: ArduinoJson v7. Note the ESP32-C3 joins 2.4 GHz networks only.

The two sketches cannot run at once — there is not enough flash for 231 KB of frames plus the TLS stack and web server in the default partition, and both want the whole display.

Appendix B Hardware diagnostic sketch

Flash this whenever something is dead and you need to know whether it is the code or the wiring. It reports the USB CDC setting, scans your wired pins for an I2C device, then tries common alternative pin pairs, then sweeps every valid pair on the C3. It finishes by beeping the buzzer three times and printing the touch pin state live so you can confirm the pad flips between 1 and 0.

It was this sketch that found the display on SDA 20 / SCL 21 rather than the 21/20 shown in the original wiring diagram — worth correcting on the diagram before it becomes a PCB.

Only GPIO 0 to 10 and 20, 21 are swept. GPIO 11 to 17 are flash, and 18 and 19 are the USB data lines; touching those would disconnect the programmer.

Appendix C Credits and licensing

The frame data was converted from the 18 GIFs in `github.com/huykhoong/esp32_dasai_mochi_clone_and_how_to`. That repository states the clips were captured from a Dasai Mochi product video, and that image rights belong to the original creator. It carries no licence file.

What that means in practice: this build is fine as a personal desk toy. It is not fine as the basis of anything you sell, exhibit commercially, or submit as original work. If you want a commercial version, the firmware here is reusable as-is — the decoder, the playback engine and the encoder tool are all original — but the artwork needs replacing. Draw your own faces, run them through `gif2mochi.py`, and everything else stays exactly the same.
